



Sede Regional Brunca

Campus Pérez Zeledón

Manual Técnico

Curso: Sistemas Operativos

Profesor: Nikole Scarlet Duarte Serrano

I Ciclo - Grupo 84

Estudiantes:

Allan Moises Calderon Ceciliano

Keylor Steven Pineda Alvarez

25 de Abril del 2026

1. Concurrency

Para esta fase del proyecto, el reto principal no era solo hacer que el servidor respondiera, sino garantizar que la información no se corrompiera cuando decenas de clientes enviaran peticiones al mismo tiempo. Basándonos en el diseño de la Fase I, implementamos la concurrencia mapeando cada componente teórico a estructuras reales en Python protegidas por mecanismos nativos de la biblioteca `threading`.

1.1 Identificación y Naturaleza de los Recursos Compartidos

En nuestro código, todo el estado del servidor vive dentro de la clase `ConcertSystem`. Durante la implementación, nos dimos cuenta de que no bastaba con identificar qué variables se compartían, sino entender cómo la estructura de datos en Python reaccionaba al acceso múltiple. Tenemos cinco recursos críticos:

La Matriz de Asientos (`self.seat_matrix`): Es el núcleo del sistema. En lugar de una sola matriz gigante, la implementamos como un diccionario donde cada llave es el ID de una zona, y su valor es una matriz bidimensional (lista de listas en Python). Cada celda guarda un string: "D" (Disponible), "R" (Reservado) o "V" (Vendido). Este es el recurso con mayor "contención" (donde más hilos pelean por acceder), por lo que lo protegimos dividiéndolo lógicamente: en lugar de bloquear todo el estadio para vender un asiento, creamos un arreglo de locks (`self.zone_lock`), uno para cada zona.

Semáforos (`self.semaphores`): Más que simples contadores, estos objetos `threading.Semaphore` son la primera barrera de defensa del servidor. Se inicializan dinámicamente multiplicando las filas por columnas de cada zona estipulada en `ZONE_CONFIG`. Actúan como un recurso compartido porque múltiples hilos hacen `acquire()` y `release()` sobre ellos constantemente. Su trabajo es frenar peticiones en seco si una zona ya se llenó, evitando que el hilo siquiera intente pelear por el lock de la matriz.

La Tabla de Reservas (`self.reservations`): Físicamente es un diccionario en memoria que guarda el ID de transacción (`tx_id`), los asientos exactos, el tiempo de creación y el TTL (180 segundos). En Python, las operaciones complejas sobre diccionarios no son seguras contra hilos (thread-safe). Si dos clientes compran al mismo tiempo, el intérprete podría sobrescribir un registro con el otro. Por eso, este recurso requirió su propio candado global: el `self.table_lock`.

La Bitácora Global (`self.event_log`): Es una simple lista donde hacemos `.append()` de los eventos. Aunque parece inofensiva, si dos hilos intentan escribir en la lista o imprimir en la consola al mismo milisegundo, los mensajes salen cortados o mezclados. Le asignamos un `self.log_lock` de muy corta duración solo para garantizar la limpieza del registro.

El Gestor de TTL (`TTLManager`): Este no es una estructura de datos, sino un hilo que corre en fondo (`daemon=True`). Lo consideramos un recurso compartido activo porque su trabajo es "despertar" cada cierto tiempo, meterse a la tabla de reservas y modificarla. Básicamente, es un cliente fantasma que compite constantemente con los hilos de los usuarios reales por el control del `table_lock` y los `zone_lock`.

1.2 Secciones Críticas y Prevención de Race Conditions

Al programar las funciones del servidor, delimitamos las secciones críticas exactamente donde la lectura o escritura de los recursos anteriores podía generar inconsistencias. A continuación,

explicamos cómo el código neutraliza los escenarios de falla (race conditions) que habíamos pronosticado:

El peligro de la doble venta (Reserva Individual): La sección crítica más importante está en `reserve_seat`. Si el cliente A y el cliente B ven libre el asiento (VIP, Fila 1, Columna 1) e intentan reservarlo juntos, ambos logran pasar el semáforo. Sin embargo, el código los obliga a hacer fila en `self.zone_lock[zone_id].acquire()`. Cuando el cliente A entra, cambia la celda a "R". Cuando el cliente A sale y el cliente B por fin entra al lock, lo primero que se topa es un `if self.seat_matrix[zone_id][row][col] != AVAILABLE`. Como ya está en "R", el hilo B aborta y se evita la doble reserva.

Inconsistencias de Aforo (Contadores vs Semáforos): Si hubiéramos usado una variable entera normal (ej. `asientos_libres -= 1`), dos hilos leyendo la variable al mismo tiempo causarían una pérdida de actualización, y el sistema creería que hay más campo del real. Al delegar esto totalmente a la clase nativa `Semaphore`, el sistema operativo se encarga de que los decrementos sean atómicos a nivel de procesador.

Estados fantasma (Confirmación vs Expiración TTL): Este era el escenario más difícil de manejar. ¿Qué pasa si la reserva expira justo en el instante en que el cliente le da click a "Confirmar Compra"? Para solucionarlo, tanto el método `confirm_purchase` como el hilo `daemon` de expiraciones tienen que adquirir primero el `table_lock`. Si el usuario gana, confirma la compra y pone la reserva en `active = False`. Si el `daemon` gana, libera el asiento y también la pone en `active = False`. Así, cuando el hilo perdedor logra entrar, ve que la transacción ya no está activa y simplemente se detiene, evitando que un asiento quede vendido en la tabla pero libre en la matriz.

Lecturas Sucias (Consulta de Disponibilidad): Incluso el método `check_availability` es una sección crítica. Si copiamos la matriz mientras otro hilo está a mitad de hacer un cambio múltiple, el cliente recibiría un mapa de asientos corrupto. Por eso, congelamos la zona un instante con `with self.zone_lock[zone_id]`: solo para hacer una copia profunda (snapshot) y la soltamos rápido para no afectar el rendimiento.

2. Sincronización

Para proteger las secciones críticas definidas anteriormente, descartamos el uso de frameworks de alto nivel que abstraen la concurrencia, cumpliendo con los requisitos del proyecto. En su lugar, implementamos mecanismos explícitos utilizando exclusivamente el módulo nativo `threading` de Python, logrando un control directo sobre el flujo de los hilos.

2.1 Mecanismos Utilizados

Implementamos dos tipos de primitivas de sincronización, cada una asignada a una capa diferente de la lógica de negocio:

Mutex (Locks de Exclusión Mutua): Utilizados a través de objetos `threading.Lock()`. Los aplicamos en dos niveles de alcance:

Alcance Medio (Por Zona): Un diccionario `self.zone_lock` que contiene un candado independiente por cada zona del concierto (VIP, Preferencial, etc.).

Alcance Global: Candados únicos para estructuras centralizadas, específicamente `self.table_lock` para el diccionario de reservas y `self.log_lock` para la bitácora de eventos.

Semáforos Contadores: Utilizados a través de objetos `threading.Semaphore()`. Creamos un diccionario `self.semaphores` donde cada zona tiene su propio semáforo, inicializado dinámicamente con su capacidad máxima exacta (filas $\times$ columnas).

En el código, estos mecanismos se manipulan explícitamente mediante llamadas a `.acquire()` y `.release()`, a menudo envueltos en bloques `try/finally` o utilizando administradores de contexto (`with lock:`) para garantizar que ningún recurso quede bloqueado en caso de una excepción.

2.2 Justificación de los Mecanismos

Las decisiones detrás de esta estrategia de sincronización responden directamente a la necesidad de evitar los cuellos de botella y las condiciones de carrera:

¿Por qué candados por zona (`zone_lock`) y no un solo candado para toda la matriz? Si hubiéramos protegido la matriz de asientos con un único lock global, el sistema habría sido seguro pero ineficiente. Un usuario comprando en "General" obligaría a un usuario comprando en "VIP" a esperar, destruyendo el propósito de la concurrencia. Por otro lado, un lock por cada asiento individual habría disparado el riesgo de deadlocks al hacer reservas múltiples. El lock por zona ofrece el balance perfecto: permite que clientes en zonas distintas operen en estricto paralelo, mientras serializa de forma segura a quienes compiten por la misma localidad.

¿Por qué un candado global para la tabla de reservas (`table_lock`)? A diferencia de la matriz, la tabla es un diccionario plano y las operaciones sobre ella (insertar un registro, cambiar `active` a `False`) son extremadamente rápidas a nivel de procesador. Usar un lock global aquí protege la estructura contra colisiones de escritura sin generar un impacto perceptible en el rendimiento del servidor.

¿Por qué separar la bitácora con su propio candado (`log_lock`)? Escribir registros y hacer `print` en consola son operaciones de I/O (Entrada/Salida) que son lentas en comparación con la manipulación de memoria. Si usáramos el `table_lock` o el `zone_lock` para proteger el log, obligaríamos a los hilos a mantener la matriz bloqueada mientras el sistema operativo imprime texto. Aislar el log con `self.log_lock` asegura que las operaciones críticas liberen los asientos lo más rápido posible.

¿Por qué usar Semáforos además de los Locks? Los locks garantizan que la modificación sea atómica, pero los semáforos funcionan como un "filtro de entrada" para el control de aforo. Si la zona VIP tiene capacidad para 40 personas, el semáforo detiene a la persona número 41 en la puerta (`acquire()` falla) antes de que intente pelear por el `zone_lock`. Además, actúan como mecanismo de señalización: cuando el gestor TTL cancela una reserva vencida, invoca `release()` sobre el semáforo, notificando de inmediato al sistema operativo que la zona vuelve a tener disponibilidad, lo cual mantiene el progreso (Liveness) del servidor.

3. Prevención de Interbloqueos (Deadlocks)

El riesgo de un interbloqueo (deadlock) no es un problema en las reservas individuales, pero se convierte en una amenaza crítica en la función `reserve_multiple`, donde un solo cliente intenta adquirir asientos en diferentes zonas simultáneamente (por ejemplo, comprando 2 entradas en Zona 1 y 2 en Zona 2).

Si dos hilos intentan adquirir los locks de estas zonas en un orden cruzado (el Hilo A bloquea la Zona 1 y pide la 2; el Hilo B bloquea la Zona 2 y pide la 1), el servidor se congelaría permanentemente.

3.1 Estrategia para evitar interbloqueos

Para que un deadlock ocurra, deben cumplirse las cuatro condiciones de Coffman. Analizando nuestra arquitectura en Python, nos dimos cuenta de que tres de ellas no podían eliminarse:

Exclusión Mutua: Intocable. Si quitamos los `zone_lock`, permitimos la doble venta.

Retención y Espera: Inevitable. Un hilo debe sostener el lock de la matriz mientras espera por el `table_lock` para registrar la transacción.

No Apropiación: El lenguaje Python no permite "robarle" o forzar la liberación de un `threading.Lock()` a otro hilo de forma segura.

Nuestra estrategia de prevención se basó enteramente en romper la cuarta condición: **La Espera Circular**. En el código, esto se logró prohibiendo que los hilos soliciten recursos al azar. Obligamos a todo el sistema a seguir una "Jerarquía Global Ascendente". En `reserve_multiple`, la clave de la estrategia está en la línea `unique_zones = sorted(zone_counts.keys())`, la cual fuerza matemáticamente al hilo a ordenar las zonas solicitadas de menor a mayor antes de intentar bloquear nada.

3.2 Orden Jerárquico de Adquisición de Recursos

Una vez que las zonas están ordenadas, el código somete a cada hilo a un flujo estricto de adquisición. Un hilo jamás puede pedir un lock de nivel 3 si no ha asegurado los del nivel 2, ni pedir la Zona 3 si ya tiene la Zona 4.

Nivel 1: Semáforos de Capacidad:Adquisición en bucle.

Antes de bloquear cualquier matriz, el hilo debe asegurar el aforo. Itera sobre las zonas ordenadas y hace `acquire()` en `self.semaphores[zone_id]`. Si en alguna zona no hay campo, se aborta inmediatamente y devuelve el aforo de las anteriores.

Nivel 2: Locks de Zona (Matriz):Exclusión mutua ascendente.

Con el aforo garantizado, el hilo bloquea físicamente las zonas usando `self.zone_lock[zone_id].acquire()`. Al hacerlo en orden ascendente (ej. Zona 0, luego Zona 2, luego Zona 3), evitamos que dos hilos formen un ciclo de espera.

Nivel 3: Lock de la Tabla de Reservas:Anidamiento de inserción.

Solo cuando el hilo posee todos los locks de zona que necesitaba, modifica la matriz a "R". Una vez modificada, adquiere el `self.table_lock.acquire()` para registrar la transacción. Este es un punto seguro porque es un único candado global, así que no hay cruce de dependencias.

Nivel 4: Lock de Bitácora:Registro final.

Tras soltar la tabla y la matriz, y solo si toda la transacción fue exitosa, el hilo adquiere `self.log_lock` para imprimir y registrar el evento.

Mecanismo de Rollback (Liberación Inversa): La adquisición en orden estricto solo es la mitad de la estrategia. Si un error ocurre en medio de una transacción múltiple (ej. un asiento de la segunda zona resulta que ya estaba reservado), el código captura la excepción y ejecuta un "Rollback". Devuelve los asientos modificados a AVAILABLE y **libera los locks en el orden exactamente inverso** al que los adquirió (reversed(acquired_locks)). Esto es vital para evitar estados inconsistentes mientras se aborta la misión.

4. Hilos y Procesos

Para este proyecto, descartamos la creación de múltiples procesos independientes (multiprocessing) en favor de un modelo basado en hilos (multithreading). La razón principal es arquitectónica: los procesos tienen espacios de memoria aislados, lo que haría extremadamente complejo compartir la matriz de asientos y el diccionario de reservas en tiempo real. Los hilos, al compartir el mismo espacio de memoria del proceso servidor, nos permiten usar threading.Lock y threading.Semaphore directamente sobre nuestras estructuras de datos.

4.1 Creación y Gestión de Hilos

Toda la gestión de hilos se realiza de forma nativa utilizando el módulo threading de Python. El sistema no utiliza un "pool" de hilos fijo, sino que los crea de forma dinámica bajo demanda, dividiéndose en dos categorías principales:

1. **Hilos Trabajadores (Worker Threads):** En el archivo servidor.py, el hilo principal se dedica exclusivamente a escuchar el puerto TCP (srv.accept()). Cuando entra una nueva conexión, el servidor delega inmediatamente el trabajo instanciando un nuevo hilo:

```
thread = threading.Thread(  
    target=handle_client,  
    args=(conn, addr, system),  
    daemon=True  
)  
thread.start()
```

Configurar estos hilos con daemon=True es una decisión crítica para la estabilidad del servidor. Si el proceso principal del servidor debe apagarse (por un crasheo o un KeyboardInterrupt), el sistema operativo destruirá automáticamente todos estos hilos hijos, evitando que queden procesos "zombies" consumiendo memoria en la máquina.

2. **Hilo Autónomo (Daemon Thread de TTL):** A diferencia de los trabajadores que reaccionan a eventos de red, el TTLManager es un hilo en segundo plano que se gestiona a sí mismo. Hereda directamente de la clase threading.Thread y ejecuta un ciclo infinito while

self._running:, durmiendo y despertando mediante `time.sleep()` para limpiar la tabla de reservas.

4.2 Relación de Hilos con Usuarios y Simulaciones

La arquitectura del sistema establece una **relación estricta de 1 a 1 entre hilos y usuarios**.

Cada vez que un cliente (usuario real o script de simulación) se conecta al servidor, recibe un hilo trabajador dedicado y exclusivo que vivirá hasta que el cliente cierre la conexión TCP.

El impacto en la simulación concurrente: Esta relación 1:1 significa que si el Generador de Carga Concurrente (exigido para la Fase III) lanza 100 peticiones de compra simultáneas, el sistema operativo levantará 100 hilos reales peleando al mismo tiempo por los locks de la matriz y el diccionario en el servidor. Este es el entorno exacto que pone a prueba la solidez de la sincronización y demuestra por qué las secciones críticas definidas en los capítulos anteriores son absolutamente necesarias para evitar la corrupción de datos.

5. Arquitectura Cliente-Servidor

Tal como definimos en el diagrama arquitectónico de la Fase I, el sistema no opera localmente como un script monolítico, sino bajo un modelo de red centralizado. El servidor mantiene los recursos compartidos, mientras que los clientes actúan como terminales que envían peticiones y procesan respuestas.

5.1 Implementación de Comunicación (Sockets TCP)

Cumpliendo con la restricción de no utilizar frameworks web (como Flask o FastAPI) ni bibliotecas de terceros que oculten la complejidad, toda la capa de red se construyó desde cero utilizando el módulo nativo `socket` de Python.

En `servidor.py`, levantamos el servidor bajo el protocolo TCP/IP, el cual es ideal para este proyecto porque es un protocolo orientado a la conexión que garantiza la entrega ordenada de los paquetes de datos sin pérdidas:

Configuración del Socket: Instanciamos `socket.socket(socket.AF_INET, socket.SOCK_STREAM)` para operar sobre IPv4 y TCP.

Alta Concurrencia de Red: Configuramos el `srv.listen(128)` para indicarle al sistema operativo que permita una cola de hasta 128 conexiones entrantes simultáneas. Esto es vital para las pruebas de estrés, ya que evita que el servidor rechace conexiones en ráfagas de alto tráfico.

Protocolo de Mensajería (JSON): Dado que los sockets en Python envían flujos de bytes puros (`b""`), establecimos un protocolo estándar de comunicación serializando la información en **JSON**. Cada cliente envía un diccionario de bytes con una "action" definida (ej. `{"action": "reserve", "zone_id": 0, "row": 1, "col": 2}`), y el servidor en `process_request` enruta la acción hacia el método correspondiente de la clase `ConcertSystem`.

6. Datos

El manejo de las reservas no solo depende de bloquear la matriz física, sino de mantener un registro lógico coherente y auditable en memoria que pueda vincular a los clientes con sus transacciones.

6.1 Estructura del Diccionario de Reservas

Implementamos la estructura temporal de los asientos en el diccionario `self.reservations` en `recursos.py`. En lugar de guardar una simple lista de asientos ocupados, cada transacción genera un UUID único (`tx_id`) que funciona como llave primaria.

El valor de esta llave es un sub-diccionario que documenta toda la vida útil de la transacción:

`zone_id`: Zona en la que se operó.

`seats`: Una lista de tuplas con las coordenadas exactas de las sillas, capaz de soportar tanto una tupla `[(row, col)]` como asientos múltiples `[(z, r, c), ...]`.

`created`: Timestamp generado con `time.time()`, esencial para que el hilo daemon pueda calcular deltas de tiempo.

`tvl`: Tiempo de vida asignado (180 segundos).

`active`: Una bandera booleana (`True` / `False`). Esta es la pieza clave que le dice al servidor si la compra fue confirmada, si caducó, o si sigue pendiente, evitando estados inconsistentes.

6.2 Protección del Acceso Concurrente a los Datos

Como ya establecimos en las secciones de concurrencia, en Python las operaciones compuestas sobre diccionarios no son atómicas.

Si el Hilo A intenta insertar la transacción 001 y el Hilo B la transacción 002 al mismo tiempo, el diccionario `self.reservations` se corrompe. Protegimos esta estructura aislando absolutamente cualquier lectura iterativa o escritura dentro de un candado de alcance global:

```
self.table_lock.acquire()
try:
    self.reservations[tx_id] = {
        "zone_id": zone_id,
        "seats": [(row, col)],
        "created": time.time(),
        "ttl": TTL_SECONDS,
        "active": True,
    }
    success = True
finally:
    self.table_lock.release() # Se libera Nivel 3
```


Este `table_lock` es exigido no solo para insertar una reserva, sino para buscarla al momento de hacer `confirm_purchase` o cuando el Gestor de TTL itera sobre los registros para limpiar la basura (`expired.append`). De esta forma, garantizamos que los datos de la reserva sean 100% seguros frente a condiciones de carrera.

7. Funcionalidad Clave

Este apartado describe la lógica de negocio que permite que el sistema pase de ser un simple contenedor de datos a un gestor transaccional robusto, asegurando que las reservas tengan un ciclo de vida controlado y seguro.

7.1 Implementación y Documentación del TTL

El **TTL (Time To Live)** es el mecanismo que garantiza la propiedad de Liveness, evitando que asientos queden bloqueados indefinidamente por usuarios que nunca finalizaron su compra. Se configuró un tiempo de expiración de 180 segundos.

El Componente Activo: `gestor_ttl.py`

Para no interrumpir la atención de los clientes, la vigilancia del tiempo se delega a una clase independiente que hereda de `threading.Thread` y corre en modo Daemon.

```
class TTLManager(threading.Thread):
    """
    Componente Gestor de Transacciones:
    Este hilo corre en segundo plano de manera autónoma (Daemon Thread).
    Su responsabilidad es revisar periódicamente si hay reservas caducadas
    y liberar los asientos/semáforos de manera segura para mantener la propiedad de Liveness.
    """
    def __init__(self, system, interval=1):
        super().__init__(daemon=True)
        self.system = system
        self.interval = interval
        self._running = True

    def run(self):
        while self._running:
            time.sleep(self.interval)
            self.system.process_expirations()

    def stop(self):
        self._running = False
```

Lógica de Expiración en `recursos.py`

El método `process_expirations` es el encargado de realizar la auditoría temporal. Es una sección crítica que requiere el `table_lock` para iterar sobre el diccionario de reservas sin que un hilo de cliente lo modifique simultáneamente.

Auditoría Temporal: Se adquiere el `table_lock`. El sistema calcula la diferencia entre el tiempo actual y el `created` de cada reserva activa. Si supera los 180s, cambia la bandera `active` a `False` para invalidarla atómicamente.

Liberación de Recursos: Para cada transacción caducada, se llama a `_release_seats`. Aquí se adquiere el `zone_lock` de la zona afectada, se revierte el estado a `AVAILABLE` en la matriz y se incrementa el semáforo contador de capacidad.

Registro y Notificación: Se genera un log con el ID de la transacción y los asientos liberados, asegurando que el administrador del sistema pueda rastrear la recuperación del inventario.

7.2 Manejo de Confirmaciones y Cancelaciones

El sistema utiliza una máquina de estados sencilla pero efectiva para manejar las transiciones finales de los asientos, asegurando que un asiento reservado solo pueda pasar a "Vendido" o volver a "Disponibile".

Confirmación de Compra

En el método `confirm_purchase`, la prioridad es la **exclusión mutua**. El hilo debe asegurar que la reserva sigue vigente antes de realizar el cambio físico en la matriz.

```
def confirm_purchase(self, tx_id):
    with self.table_lock:
        res = self.reservations.get(tx_id)
        if not res or not res["active"]:
            return False, "Transacción no válida o ya procesada"
        res["active"] = False

        multiple = res.get("multiple", False)
        seats = res["seats"]

        if multiple:
            for z in sorted(set(s[0] for s in seats)):
                with self.zone_lock[z]:
                    for s in seats:
                        if s[0] == z:
                            self.seat_matrix[s[0]][s[1]][s[2]] = SOLD
        else:
            row, col = seats[0]
            with self.zone_lock[res["zone_id"]]:
                self.seat_matrix[res["zone_id"]][row][col] = SOLD

        if multiple:
            seats_str = ", ".join(f"Z{s[0]} F{s[1]}C{s[2]}" for s in seats)
        else:
            seats_str = f"Z{res['zone_id']} F{seats[0][0]}C{seats[0][1]}"

        self._log(f"[COMPRA CONFIRMADA] TX: {tx_id} | Asientos: {seats_str}")
        return True, None
```

8. Pruebas y Validación

La validación del sistema se centró en demostrar que los mecanismos de sincronización (Locks y Semáforos) soportan condiciones de estrés real sin violar la propiedad de Safety.

8.1 Evidencia de Ejecución Concurrente

Se diseñó un script de prueba que lanza múltiples hilos de forma simultánea atacando el mismo recurso compartido. En los logs obtenidos (logs_prueba_concurrente.txt), se observa el comportamiento esperado del planificador.

Escenario de Prueba	Acción Realizada	Resultado Observado en Logs
Colisión Directa	10 hilos intentan reservar el asiento Z0 F0 C0 al mismo tiempo.	1 hilo obtiene éxito; 9 hilos reciben error "Asiento no disponible".
Reserva Múltiple	2 hilos intentan reservar rangos que se traslapan.	El primer hilo bloquea la zona y reserva todo el rango; el segundo falla atómicamente.
Carrera TTL	Se intenta confirmar una compra en el segundo 179.9.	El log muestra la confirmación exitosa y el hilo de TTL ignora la transacción al verla inactiva.

8.2 Pruebas de Estrés (Generador de Carga)

Siguiendo el requerimiento de la Fase III, se implementó un generador de carga capaz de parametrizar cientos de solicitudes.

Durante las ejecuciones de **100 y 200 solicitudes concurrentes**, el servidor mantuvo una integridad del 100% en la matriz de asientos:

Cero dobles ventas: Ningún asiento terminó con dos dueños diferentes.

Coherencia de Semáforos: Al finalizar todas las transacciones y expiraciones, los semáforos regresaron a su valor original de capacidad, demostrando que no hubo fugas de recursos (resource leaks).

Estabilidad del Servidor: El uso de hilos dinámicos (daemon=True) permitió que el servidor no se colgara incluso ante ráfagas de conexiones TCP simultáneas.

9. Justificación Técnica

El diseño y la implementación de este sistema concurrente no se basaron en lograr que el código simplemente compilara, sino en resolver el problema clásico de la gestión de recursos compartidos bajo alta demanda transaccional.

A continuación, justificamos las decisiones técnicas más críticas que tomamos para garantizar la integridad del sistema:

9.1 Control Explícito vs. Frameworks de Alto Nivel

Decidimos implementar toda la capa de red con socket y la concurrencia con threading nativo, rechazando deliberadamente cualquier framework moderno (como Django o FastAPI) o bases de datos con control transaccional integrado. Esta decisión técnica asegura que nosotros, y no una caja negra, controlamos exactamente en qué ciclo de CPU se adquiere y se libera un recurso, demostrando una comprensión profunda de la exclusión mutua.

9.2 Granularidad de los Locks (Rendimiento vs. Seguridad)

Comprendimos que el mayor riesgo del sistema era la "Doble Venta" (violación de Safety). La solución más fácil habría sido colocar un único candado global (`lock_estadio_completo`) para cualquier reserva. Sin embargo, esto habría convertido nuestro servidor concurrente en un sistema secuencial (un cuello de botella masivo). La decisión de implementar un arreglo de candados `self.zone_lock[zone_id]` fue una decisión calculada: garantizamos la exclusión mutua estricta para los hilos que compiten por el mismo recurso (ej. dos usuarios peleando por la Zona VIP), pero permitimos ejecución 100% paralela para hilos que operan en zonas distintas.

9.3 Eliminación de la Espera Circular

El problema más complejo de modelar fue la reserva múltiple en diferentes zonas, donde el riesgo de un Deadlock era inminente si dos hilos cruzaban sus peticiones. La decisión de forzar mediante código un ordenamiento jerárquico ascendente (`unique_zones = sorted(zone_counts.keys())`) antes de permitir cualquier `acquire()` demuestra nuestra comprensión de las Condiciones de Coffman. Al eliminar la posibilidad matemática de una espera circular, salvaguardamos la propiedad de Liveness del servidor bajo cualquier nivel de estrés.

10. Calidad del Entregable

Para asegurar que este proyecto sea mantenible, auditable y fácil de evaluar, estructuramos tanto la documentación como el código bajo estrictos estándares de orden.

10.1 Estructura del Código

El código fuente no es un script monolítico. Se aplicó el principio de separación de responsabilidades dividiendo el backend en tres módulos lógicos:

1. **recursos.py**: El núcleo del sistema operativo. Aísla toda la lógica de los recursos compartidos, las secciones críticas, los arreglos de Locks y los Semáforos.
2. **servidor.py**: Aísla la capa de transporte (Sockets TCP), la escucha en el puerto 9090, el enrutamiento de peticiones JSON y la instanciación dinámica de los hilos de cliente (Worker Threads).
3. **gestor_ttl.py**: Aísla el hilo daemon que trabaja en segundo plano, evitando mezclar la lógica de limpieza de memoria con la atención al cliente.

10.2 Documentación Interna (Comentarios)

El código fue rigurosamente comentado, no para explicar qué hace Python, sino para documentar las decisiones de concurrencia. Cada sección crítica está etiquetada indicando el nivel jerárquico del lock (NIVEL 1, NIVEL 2, etc.), advirtiendo sobre las propiedades de Safety y Liveness, y justificando las acciones de mitigación de Deadlocks (como los bloques de Rollback inverso) directamente en las líneas donde ocurren los bloqueos.